Real-Time Tennis Oddsmaking and Integrity Monitoring

A Big Data System Architecture and Implementation Plan

Revised Edition — All Architectural Gaps Addressed

Team Members: Add Names and NetIDs

April 28, 2026

Revision Note. This document incorporates all 24 architectural gaps identified in the original proposal review. New or modified content is present throughout every section. Gap annotations (**CRITICAL**, **IMPORTANT**, **MINOR**) mark each addition so reviewers can locate changes quickly.

Contents

1 Purpose of This Document

This document defines the final architecture, implementation plan, data contracts, pipeline design, modeling approach, testing strategy, and presentation plan for the big data project titled *Real-Time Tennis Oddsmaking and Integrity Monitoring*. The goal is to provide a single reference that guides development, team coordination, cloud deployment, report writing, and final demonstration.

The project follows a practical Lambda-style architecture deployed using two Amazon EC2 instances and an Amazon S3 data lake. EC2 Instance 1 acts as the ingestion, orchestration, streaming, batch processing, and machine learning node. EC2 Instance 2 acts as the serving and application node. Amazon S3 is the durable data lake storing raw data, cleaned data, feature tables, model artifacts, baseline profiles, benchmark outputs, and final report assets.

2 Project Thesis and Big Data Justification

2.1 Project Thesis

The project builds a real-time analytics system for professional tennis supporting two related use cases from the same underlying event stream. First, it computes live win probabilities for simulated tennis matches using point-by-point streaming events. Second, it monitors match-integrity risk by comparing live player behavior against historical baselines and flagging statistically unusual deviations.

The central thesis is that live sports prediction and integrity monitoring can be unified into one data engineering system because both require granular event streams, historical baselines, rolling features, low-latency scoring, and serving layers that expose results to end users.

2.2 Why This Is a Data Engineering Problem

This project is primarily a data engineering problem because the system must move, transform, store, and serve data across multiple stages. The core work is not only building a machine learning model, but building the infrastructure around the model. The system must ingest raw historical files, prepare replayable event streams, process streaming data, build feature tables, train models, store outputs, and serve results through APIs and dashboards.

Core data engineering challenges include:

- maintaining a clean data lake layout in S3;
- converting historical point-level data into live replay streams;
- processing Kafka events with Spark Structured Streaming;
- generating batch features and player baselines;
- coordinating offline and online pipelines via Airflow DAG dependencies;
- persisting serving-ready outputs in PostgreSQL and optional Redis;
- exposing results through FastAPI and React.

2.3 Why This Is a Big Data Project Rather Than a Small Local Script

A small script could train a static model on a subset of tennis matches but would not demonstrate the distributed and system-level requirements of this project. This project involves high-volume historical processing, synthetic stream amplification, real-time event ingestion, stateful streaming computations, batch retraining, and separate serving responsibilities.

The big data nature comes from:

- **Volume:** historical data expanded into a larger synthetic archive;
- **Velocity:** point events replayed as live streams through Kafka;
- **Variety:** match-level, point-level, player-level, model, and risk-event data coexist;
- **Complexity:** rolling features, baselines, anomaly scores, and live odds computed consistently;
- **Deployment:** multiple services across two cloud instances and S3.

2.4 Target Scale Envelope

- Replay hundreds to thousands of simulated matches depending on EC2 resources;
- process point events through Kafka topics partitioned by match identifier;
- store raw and processed data in Parquet format in S3;
- maintain low-latency serving of current match states through PostgreSQL and optional Redis;
- expose dashboards that refresh frequently enough to feel real-time for demonstration.

2.5 Numerical Benchmark Targets

Metric	Target
Streaming throughput	At least 1,000 point events per second for MVP; higher as stretch goal
End-to-end latency	Under 5 seconds from event production to dashboard visibility
Batch processing	Complete feature generation on selected dataset within demo window
Model output latency	Streaming inference completed within each Spark micro-batch
Serving read latency	API responses under 500 ms in normal demo conditions
Dashboard refresh	2–5 second refresh interval for live views

Table 1: Benchmark targets

3 Executive Architecture Decision

3.1 Final Architectural Choice

The final architecture is a practical Lambda-style architecture using:

- Amazon S3 as the central data lake;
- EC2 Instance 1 as the processing node;
- EC2 Instance 2 as the serving and application node;
- Apache Kafka for live event ingestion;
- Apache Spark for batch processing and streaming analytics;
- Apache Airflow for workflow orchestration;
- PostgreSQL for serving current match, odds, and risk-event data;
- optional Redis for low-latency latest-state caching;
- FastAPI for backend APIs;
- React for the frontend dashboard.

3.2 Why This Choice Was Made

This architecture balances ambition, cost control, and implementation feasibility. Fully managed services such as Amazon MSK, Amazon EMR, and managed OpenSearch would be closer to a production cloud architecture but are cost-prohibitive for a student project. A purely local system would be cheaper but would not demonstrate realistic cloud deployment.

The two-EC2 architecture provides a strong middle ground: real services running in AWS, separated processing from serving, S3 as a real cloud data lake, and manageable operational complexity.

4 System Overview

4.1 High-Level Components

Component	Responsibility
S3 Data Lake	Stores raw, cleaned, features, models, baselines, logs, and benchmark outputs
EC2 Instance 1	Runs ingestion, Kafka, Spark, ML workflows, and Airflow
EC2 Instance 2	Runs PostgreSQL, optional Redis, FastAPI, React, and optional Nginx
Kafka	Buffers and distributes simulated live point events
Spark Batch	Builds curated datasets, features, baselines, and model training inputs
Spark Streaming	Processes live point events, computes rolling features, and performs online scoring
Airflow	Schedules and monitors batch workflows and model publication jobs
PostgreSQL	Stores serving-ready current match state, odds history, and risk events
Redis	Optional cache for latest match state and live dashboard reads
FastAPI	Provides API endpoints to the dashboard
React	Provides the user-facing live odds and risk exploration interface

Table 2: High-level system components

4.2 Canonical Flow

1. Raw tennis datasets are uploaded to S3.
2. EC2 Instance 1 reads raw data from S3.
3. Spark batch jobs clean data and build curated datasets.
4. Feature engineering jobs create offline feature tables and player baselines.
5. Model training jobs train odds and risk models.
6. Model artifacts and baselines are published back to S3 via a validated two-phase publish.
7. A replay producer reads prepared point sequences and sends events to Kafka.
8. Spark Structured Streaming consumes Kafka events.
9. Streaming jobs compute rolling features and apply latest models.
10. Current odds and risk outputs are written to PostgreSQL and optionally Redis on EC2 Instance 2.
11. FastAPI exposes serving data through API endpoints.
12. React dashboard displays live odds, match drill-downs, risk events, and benchmark summaries.

5 Scope Definition

5.1 What Will Be Real

- S3 data lake layout with zone manifests;
- Kafka-based event replay with explicit topic configuration;
- Spark batch jobs for cleaning and feature generation;
- Spark streaming job for live processing with dead-letter handling;
- baseline odds model;
- baseline anomaly/risk scoring model;
- PostgreSQL serving schema with indexes and archival;
- FastAPI backend with bearer token authentication;
- React dashboard;
- benchmark scripts and result exports.

5.2 What May Be Simplified

- Number of features may be reduced to a practical MVP set;
- model choices may use scikit-learn or Spark MLlib depending on complexity;
- Redis may be optional if PostgreSQL performance is sufficient;
- Airflow initially orchestrates one MVP DAG for the offline pipeline;
- streaming scale may be lower than a full production sports-betting workload.

5.3 What Will Not Be Claimed

- Real betting odds suitable for financial wagering;
- anomaly flags as proof of match-fixing or misconduct;
- synthetic replay exactly matching live professional tennis operations;
- two-instance deployment at production scale;
- legally or commercially validated models.

6 Repository Structure

```
tennis-big-data-project/
README.md
.github/
  workflows/
    ci.yml                # linting, unit tests, compose-validate
docker-compose.processing.yml
docker-compose.serving.yml
docker-compose.dev.yml    # single-machine dev with shared bridge
   network
.env.example
data/
  sample/
contracts/
  point_event_schema.json     # enforced on raw ingestion
  match_state_schema.json
  risk_event_schema.json
infra/
  ec2/
    kafka_setup.sh        # topic creation with explicit partition
       config
```

```
    spark_defaults.conf  # Iceberg catalog + micro-batch tuning
  s3/
    create_layout.sh     # creates prefixes + zone_metadata.json
  iam/
    ec2-processing-role-policy.json
    ec2-serving-role-policy.json
  nginx/
producer/
  replay_producer.py
  config.py
spark/
  batch/
    historical_ingestion.py
    curated_build.py
    feature_build.py
    baseline_build.py
    train_models.py
    publish_model.py     # two-phase validated model publish
  streaming/
    streaming_processor.py
    sinks.py
    scoring.py
    dead_letter.py       # malformed-message handling
airflow/
  dags/
    nightly_batch_pipeline.py   # A->B->C->D->E->publish->F
    replay_demo_pipeline.py
api/
  app/
    main.py
    middleware/
      auth.py            # bearer token check
    routes/
    db/
    schemas/
frontend/
  src/
    pages/
    components/
    services/
sql/
  serving_schema.sql
  indexes.sql            # idx_odds_history_match_ts + others
  archival.sql           # pg_cron archival job
tests/
  unit/
  integration/
  e2e/
reports/
  benchmark_results/
```

7 Technology Decisions

7.1 Core Stack

Layer	Technology	Reason
Cloud Storage	Amazon S3	Durable data lake and artifact store
Lakehouse Table Format	Apache Iceberg (Hadoop catalog)	Schema evolution, snapshots, incremental reads on curated/features zones
Compute	Two Amazon EC2 instances	Cost-controlled cloud deployment
Streaming Ingestion	Apache Kafka	Event buffering and replay simulation
Batch Processing	Apache Spark	Scalable feature engineering
Streaming Processing	Spark Structured Streaming	Stateful live event processing
Orchestration	Apache Airflow	Scheduled workflows with explicit DAG dependencies
Serving Database	PostgreSQL	Reliable relational serving store
Cache	Redis (optional)	Fast latest-state reads
Backend	FastAPI	Lightweight Python API layer
Frontend	React	Interactive dashboard
Containerization	Docker Compose	Reproducible deployment
Secrets Management	AWS SSM Parameter Store	Shared secrets across both EC2 instances

Table 3: Core technology stack

7.2 Iceberg Catalog Configuration

IMPORTANT GAP — Gap #09 addressed

Iceberg requires a catalog backend. For this two-EC2 student deployment the **Hadoop catalog backed by S3** is used. No external catalog service is required. The following configuration must be present in `infra/ec2/spark_defaults.conf` and referenced in `docker-compose.processing.yml`:

```
spark.sql.extensions=org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensi
spark.sql.catalog.iceberg=org.apache.iceberg.spark.SparkCatalog
spark.sql.catalog.iceberg.type=hadoop
spark.sql.catalog.iceberg.warehouse=s3://tennis-big-data/iceberg/
spark.sql.defaultCatalog=iceberg
```

Listing 1: spark-defaults.conf — Iceberg Hadoop catalog

Iceberg scope. Iceberg is applied only to the `cleaned/` and `features/` zones. The `raw/` zone remains plain S3 objects. The `baselines/` and `models/` zones use versioned Parquet files, not Iceberg tables, because they are written once per training run rather than updated incrementally.

7.3 Explicit Technology Rejections

- **Amazon MSK:** rejected due to cost for student version;
- **Amazon EMR:** optional for stretch benchmarking only;
- **Elasticsearch/OpenSearch:** replaced by PostgreSQL risk-event serving;
- **Cassandra:** replaced by PostgreSQL and optional Redis;

- **Kubernetes:** Docker Compose sufficient for two EC2 instances.

8 Environment Split

8.1 Local Development

Gap #23 addressed. A dedicated `docker-compose.dev.yml` runs all services on a single machine using a shared Docker bridge network. This eliminates configuration drift between developers and removes the need to manage two sets of private IPs during local testing. Elastic IPs are applied to both EC2 instances so private IPs remain stable across reboots.

Local development responsibilities:

- writing and testing Spark transformations on samples;
- testing producer payloads;
- developing FastAPI endpoints;
- developing React pages;
- running schema validation and unit tests.

8.2 Cloud Deployment

EC2 Instance 1 — Processing Node. Kafka, producer, Spark, Airflow, ML pipelines. Recommended type: `m5.xlarge` (4 vCPUs, 16 GB RAM).

CRITICAL GAP — Gap #10 addressed

EC2-1 runs Kafka, Spark, and Airflow concurrently on 16 GB RAM. To prevent OOM during model training:

- Mount a 30 GB EBS scratch volume at `/mnt`; set `spark.local.dir=/mnt/spark-temp` in `spark-defaults.conf`.
- Configure 8 GB swap: `sudo fallocate -l 8G /swapfile && sudo mkswap /swapfile && sudo swapon /swapfile`.
- When running Pipeline E (model training), **pause the Kafka consumer and replay producer** to free RAM. Document this in the Airflow DAG via task dependencies.

EC2 Instance 2 — Serving and Application Node. PostgreSQL, optional Redis, FastAPI, React, optional Nginx. Recommended type: `t3.large` or `m5.large`.

8.3 Networking and Security Group Setup

Both EC2 instances must be in the same AWS region, VPC, and preferably the same private subnet. EC2-1 communicates with EC2-2 over private IP addresses only.

CRITICAL GAP — Gap #05 addressed: IAM role separation

Two separate IAM roles must be created and documented in `infra/iam/`:
ec2-processing-role (attached to EC2-1):

- `s3:GetObject`, `s3:PutObject`, `s3:DeleteObject` on all prefixes;
- `ssm:GetParameter` on `/tennis/*`;
- `ec2:DescribeInstances` for instance self-identification.

ec2-serving-role (attached to EC2-2):

- `s3:GetObject` on `models/` and `baselines/` prefixes **only**;

- `ssm:GetParameter` on `/tennis/*`;
- **No S3 write permissions** — EC2-2 must never write to the data lake.

EC2-2 security group must allow PostgreSQL (5432) and Redis (6379) traffic only from EC2-1's private security group. Public inbound must be restricted to SSH from developer IPs and port 80/443 or 8000 for the demo.

8.4 Shared Secrets Management

IMPORTANT GAP — Gap #18 addressed

All shared secrets (`POSTGRES_PASSWORD`, `REDIS_HOST`, `API_KEY`, etc.) are stored in **AWS Systems Manager Parameter Store** as `SecureString` values under the `/tennis/` prefix. Both EC2 instances read from SSM at startup:

```
export POSTGRES_PASSWORD=$(aws ssm get-parameter \
  --name /tennis/postgres_password \
  --with-decryption \
  --query Parameter.Value \
  --output text)
```

This prevents secrets from appearing in git, `.env` files synced between instances, or EC2 instance metadata. The `.env.example` file references SSM parameter names, not values.

9 Data Contracts

9.1 Point Event Contract

```
{
  "event_id":        "string",
  "match_id":        "string",
  "event_ts":        "timestamp (UTC ISO-8601)",
  "tournament":      "string",
  "surface":         "string",
  "player_a":        "string",
  "player_b":        "string",
  "server":          "string",
  "receiver":        "string",
  "point_winner":    "string",
  "set_number":      1,
  "game_number":     4,
  "point_number":    23,
  "score_context":   "string",
  "serve_number":    1,
  "rally_length":    6,
  "is_break_point": false,
  "is_double_fault":false,
  "is_ace":          false
}
```

Listing 2: Point event Kafka message schema

9.2 Current Match State Contract

```
{
  "match_id":              "string",
  "updated_at":            "timestamp (UTC ISO-8601)",
  "player_a":              "string",
  "player_b":              "string",
  "player_a_win_prob":     0.62,
  "player_b_win_prob":     0.38,
  "momentum_player":       "string",
  "momentum_score":        0.14,
  "set_score":             "string",
  "game_score":            "string",
  "points_played":         87,
  "status":                "live"
}
```

Listing 3: Current match state serving contract

9.3 Risk Event Contract

```
{
  "risk_event_id":    "string",
  "match_id":         "string",
  "event_ts":         "timestamp (UTC ISO-8601)",
  "player":           "string",
  "opponent":         "string",
  "risk_score":       0.83,
  "risk_level":       "high",
  "primary_signal":   "serve_win_rate_drop",
  "feature_deviation_summary": {
    "serve_win_rate_delta":     -0.31,
    "rally_length_delta":        4.2,
    "pressure_point_delta":     -0.22
  },
  "explanation": "Current rolling serve performance is far below
     historical baseline."
}
```

Listing 4: Risk event contract

9.4 Schema Governance Rules

- Every event must include a unique `event_id`.
- Every event must include `match_id` for partitioning and joins.
- Timestamp fields must use UTC ISO-8601.
- New fields must be backward compatible where possible.
- Null handling rules must be defined for every feature.
- Schema changes must be documented and increment the schema version.

IMPORTANT GAP — Gap #24 addressed: schema enforcement on raw ingestion

Raw zone files are plain S3 objects (no Iceberg). To prevent mixed-schema Parquet files after pipeline changes, `historical_ingestion.py` validates every incoming file against `contracts/point_event_schema.json` before writing. Any column additions require a schema version bump (`v1`, `v2`, ...) and a full raw-zone re-ingest. Failed validation routes files to `s3://tennis-big-data/raw/quarantine/`.

10 Data Lake Design

10.1 Zones

Zone	Format	Description
`raw/`	CSV / plain S3	Original source files; schema-validated on ingest
`cleaned/`	Parquet (Iceberg)	Standardized, validated records
`features/`	Parquet (Iceberg)	Offline feature tables
`baselines/`	Parquet (versioned)	Player baseline profiles
`models/`	Pickle/MLlib (versioned)	Trained model artifacts
`stream_archive/`	Parquet	Archived point events and streaming outputs
`results/`	JSON/CSV	Benchmark and report outputs

Table 4: S3 data lake zones

10.2 Storage Layout

```
s3://tennis-big-data/
  raw/
    matches/
    points/
    quarantine/           # schema-validation failures
    zone_metadata.json    # zone manifest
  cleaned/                # Iceberg-managed
    matches/
    points/
    zone_metadata.json
  features/               # Iceberg-managed
    match_features/
    point_features/
    zone_metadata.json
  baselines/
    player_baselines/
    zone_metadata.json
  models/
    odds/
      v1/, v2/, ...
      latest.json         # two-phase publish pointer
    risk/
      v1/, v2/, ...
      latest.json
    zone_metadata.json
  stream_archive/
    point_events/
    scored_events/
    zone_metadata.json
  results/
    benchmarks/
    reports/
    zone_metadata.json
  iceberg/                # Iceberg Hadoop catalog metadata root
```

MINOR GAP — Gap #19 addressed: zone manifests

Each zone root contains a `zone_metadata.json` file written by `infra/s3/create_layout.sh`. The file records: `zone_name`, `managed_by` (`iceberg/parquet/plain`), `owner`, `created_at`, `description`. This prevents confusion when team members browse the bucket.

10.3 Partitioning

- By year for historical match data;
- by tournament or surface for selected feature tables;
- by replay date for stream archive data;
- by model version for artifacts.

10.4 File Formats

- **CSV:** raw data ingestion only;
- **Parquet:** physical storage for cleaned data, features, stream archives;
- **Apache Iceberg:** table format for curated and feature zones;
- **JSON:** event contracts, zone manifests, model registry pointers;
- **Pickle/MLlib/Joblib:** model artifacts depending on framework.

11 Lake vs. Serving Responsibilities

11.1 Data Lake Responsibilities

S3 is responsible for durable, reproducible storage. It holds the full historical record, intermediate datasets, offline features, models, baselines, and benchmark evidence.

11.2 Serving Layer Responsibilities

The serving layer holds dashboard-ready data. PostgreSQL stores current match states, odds history, risk events, model versions, and benchmark summaries. Redis may store the latest state for low-latency reads.

11.3 Design Rule

The data lake is the source of truth. The serving layer is optimized for application access. If serving tables are corrupted or deleted, they must be fully rebuildable from S3 and pipeline outputs.

12 Serving Data Model

12.1 PostgreSQL Tables

```
CREATE TABLE matches (
  match_id    TEXT PRIMARY KEY,
  player_a    TEXT NOT NULL,
  player_b    TEXT NOT NULL,
  tournament  TEXT,
  surface     TEXT,
  status      TEXT,
```

```
  created_at   TIMESTAMP,
  updated_at   TIMESTAMP
);

CREATE TABLE current_match_state (
  match_id            TEXT PRIMARY KEY REFERENCES matches(match_id),
  updated_at          TIMESTAMP NOT NULL,
  player_a_win_prob   DOUBLE PRECISION,
  player_b_win_prob   DOUBLE PRECISION,
  momentum_player     TEXT,
  momentum_score      DOUBLE PRECISION,
  set_score           TEXT,
  game_score          TEXT,
  points_played       INTEGER
);

CREATE TABLE odds_history (
  id                  BIGSERIAL PRIMARY KEY,
  match_id            TEXT REFERENCES matches(match_id),
  event_ts            TIMESTAMP NOT NULL,
  player_a_win_prob   DOUBLE PRECISION,
  player_b_win_prob   DOUBLE PRECISION,
  momentum_score      DOUBLE PRECISION,
  points_played       INTEGER
);

CREATE TABLE risk_events (
  risk_event_id  TEXT PRIMARY KEY,
  match_id       TEXT REFERENCES matches(match_id),
  event_ts       TIMESTAMP NOT NULL,
  player         TEXT,
  opponent       TEXT,
  risk_score     DOUBLE PRECISION,
  risk_level     TEXT,
  primary_signal TEXT,
  explanation    TEXT
);

CREATE TABLE model_versions (
  model_version  TEXT PRIMARY KEY,
  model_type     TEXT,
  artifact_path  TEXT,
  trained_at     TIMESTAMP,
  metrics_json   JSONB
);
```

Listing 5: serving_schema.sql

CRITICAL GAP — Gap #03 addressed: indexes and archival

Indexes (indexes.sql):

```
-- Dashboard reads filter by match_id and sort by event_ts DESC
CREATE INDEX idx_odds_history_match_ts
  ON odds_history (match_id, event_ts DESC);

CREATE INDEX idx_risk_events_match_ts
```

```
  ON risk_events (match_id, event_ts DESC);

CREATE INDEX idx_risk_events_level
  ON risk_events (risk_level, event_ts DESC);
```

Archival job (archival.sql — scheduled via pg_cron every 15 minutes):

```
-- Retain only the last 2 hours of odds_history rows
DELETE FROM odds_history
WHERE event_ts < NOW() - INTERVAL '2 hours';

-- Archive risk events older than 24 hours to S3 before deletion
-- (handled by Airflow Pipeline I before DELETE runs)
DELETE FROM risk_events
WHERE event_ts < NOW() - INTERVAL '24 hours';
```

JDBC batch size must be set in `streaming_processor.py`:

```
odds_df.write \
  .format("jdbc") \
  .option("url", POSTGRES_URL) \
  .option("dbtable", "odds_history") \
  .option("batchsize", "500") \
  .mode("append") \
  .save()
```

12.2 Redis Keys (Optional)

```
match:{match_id}:latest_state     TTL = 30s
match:{match_id}:latest_odds      TTL = 30s
live_matches                      TTL = 10s
system:latest_benchmark           TTL = 300s
```

IMPORTANT GAP — Gap #15 addressed: odds_history size estimate

For 10 concurrent matches at a 2-second micro-batch trigger over 90 minutes of demo time, `odds_history` accumulates approximately $10 \times 2{,}700 = 27{,}000$ rows. This is manageable but must be a conscious design decision. The pg_cron archival job above bounds the table to the last 2 hours. Live dashboard reads use `current_match_state` (one row per match) rather than querying `odds_history` for the latest value.

12.3 Serving Retention Strategy

PostgreSQL retains all demo-period current states, odds histories within the 2-hour rolling window, and risk events within 24 hours. Redis retains only latest values and is treated as disposable cache. Long-term archives remain in S3.

13 Synthetic Data Strategy

13.1 Core Principle

Synthetic data should amplify scale without inventing unrealistic conclusions. Synthetic replay preserves the structure of real tennis point sequences while allowing many simulated matches to run concurrently.

13.2 Allowed Methods

- Replaying historical point sequences with new synthetic match identifiers;
- shifting event timestamps to simulate live progression;
- sampling matches by tournament, surface, or player;
- modest perturbation of non-critical metadata for scale testing;
- duplicating replay streams for controlled throughput benchmarks.

13.3 Disallowed Methods

- Inventing risk labels and presenting them as real evidence;
- claiming synthetic anomalies are confirmed match-fixing;
- corrupting tennis scoring rules;
- using future points to compute current live features;
- producing models that leak target outcomes into live prediction features.

IMPORTANT GAP — Gap #16 addressed: anomaly training strategy

The risk model must be trained without invented anomaly labels. The approach is fully **unsupervised**:

1. Compute historical player baselines (mean and standard deviation of each feature per player per surface) from the full historical dataset.
2. During streaming, compute z-scores of current rolling features against the stored baseline.
3. Flag events where $|z| > 2.5$ on two or more key signals simultaneously as `risk_level = high`.
4. Optionally train an Isolation Forest on historical feature vectors to score global anomalousness alongside the z-score approach.

This approach requires no labeled anomaly examples and is not susceptible to label fabrication. Document this in `README.md` under the Modeling section.

13.4 Documentation Requirement

Every synthetic generation method must be documented with: input source, transformation rule, output location, intended purpose, and limitations.

14 Modeling Plan

14.1 Predictive Odds Model

The odds model predicts each player's probability of winning given current match state. For the MVP the model may use Logistic Regression, Random Forest, or Gradient Boosted Trees.

Inputs: current score context, serve win rate, return points won, recent point streak, surface, player baseline strength, rolling momentum features.

Outputs: player A win probability, player B win probability, model version metadata.

14.2 Risk / Anomaly Model

The risk model identifies unusual deviations between current player behavior and historical baselines using z-score deviation and optional Isolation Forest (see Section 13.2 above).

Risk signals: serve win rate significantly below baseline, double faults above baseline, unusual drop in pressure-point performance, abnormal rally length pattern, sudden momentum collapse.

14.3 Why Spark MLlib Is Not the Sole Path

Spark MLlib is useful for distributed training but the project may use scikit-learn or another Python ML library for MVP if the selected dataset is manageable. Spark remains central for distributed feature generation and streaming analytics.

14.4 Model Governance Rules

- Every model artifact must have a version identifier.
- Every model must store its training timestamp.
- Evaluation metrics must be saved to `benchmark_results/model_eval.json`.
- The streaming job must log which model version was used.
- Risk scores must be described as statistical signals, not proof of misconduct.

CRITICAL GAP — Gap #13 addressed: two-phase model publish with validation gate

`publish_model.py` implements a two-phase publish to prevent corrupt or incomplete model artifacts from reaching the streaming job:

1. Train and save model to `s3://tennis-big-data/models/odds/staging/`.
2. Load the staged artifact and score a fixed test fixture (`data/sample/model_validation_fixture.parquet`).
3. Assert: `AUC >= 0.55` and `brier_score <= 0.30`. If either fails, abort — do not update `latest.json`.
4. On success, copy artifact from `staging/` to `v{N}/` and atomically overwrite `latest.json` with the new version path.

S3 object versioning is enabled on the `models/` prefix as a recovery fallback.

IMPORTANT GAP — Gap #20 addressed: model evaluation thresholds

The following minimum thresholds gate model publication. They are deliberately conservative for a synthetic dataset:

- Odds model: AUC ≥ 0.55, Brier score ≤ 0.30.
- Risk model: precision on held-out injected anomalies ≥ 0.50 (if labeled test set available); otherwise, validate that z-score distribution on the test set matches expected Gaussian properties.

These values are logged to `results/benchmarks/model_eval.json` on every training run.

During streaming, the processor loads the latest approved model at startup and refreshes only at controlled boundaries. Model artifacts are identified by `latest.json`. A version switch is logged when it occurs.

15 Feature Design

15.1 Feature Families

Family	Example Features
Serve Dynamics	first serve %, ace rate, double fault rate, serve points won
Return and Rally	return points won, avg. rally length, rally length trend
Pressure and Clutch	break-point conversion, break-point save rate, deuce performance
Momentum and Fatigue	recent points won, streak length, cumulative rally load, momentum index
Contextual	surface, tournament, round, player baseline, opponent strength

Table 5: Feature families

15.2 Rules for Feature Correctness

CRITICAL GAP — Gap #07 addressed: point-in-time correctness

Batch features (Pipeline C):

- Sort all events by `event_ts` before computing any rolling window.
- Apply a **temporal train/test split**: all matches completed before date T form the training set; matches after T form the test set. Never allow future match data to appear in training features.
- Use `Window.rowsBetween(Window.unboundedPreceding, -1)` in Spark window functions so the current row's own outcome is never included in its own feature.
- Document the split date and window boundaries in `features/feature_catalog.md`.

Streaming features (live):

- Streaming features may only use events with `event_ts` $\leq$ `current_event_ts`.
- Watermark of 30 seconds ensures late events are bounded (see Section 18.3).
- Rolling windows use `withWatermark` before any stateful aggregation.

- Null values must be explicitly handled (fill with baseline mean or zero as documented).
- Feature definitions must be documented and unit-tested.

16 Data Quality Plan

16.1 Core Data Quality Checks

- Required fields are present;
- event timestamps are parseable and in UTC;
- match IDs are non-null;
- `point_winner` is one of the valid players;
- set/game/point numbers are non-negative;
- duplicate event IDs are removed or ignored;
- invalid scoring states are flagged.

16.2 DQ Failure Policy

Records failing non-critical checks are quarantined at `s3://tennis-big-data/raw/quarantine/`. Records failing critical checks are not published to Kafka or serving tables. Rejected record counts are logged per pipeline run.

17 Pipelines

17.1 Pipeline A: Historical Ingestion

Loads raw source files into S3, validates against `contracts/point_event_schema.json`, and writes zone metadata. Triggered by Airflow S3KeySensor or CLI script `ingest.sh`.

IMPORTANT GAP — Gap #08 addressed: ingestion trigger

Pipeline A has a defined entry point. Option 1 (recommended): an Airflow `S3KeySensor` watching `s3://tennis-big-data/raw/points/`. Option 2: a bash script `ingest.sh` that syncs a local data folder and then triggers the Airflow DAG via the CLI (`airflow dags trigger nightly_batch_pipeline`). Both options are documented in `README.md` under Quick Start.

17.2 Pipeline B: Curated Dataset Build

Reads raw S3 data, standardizes columns, normalizes types, validates records, and writes cleaned Iceberg tables.

17.3 Pipeline C: Feature Build

Builds batch feature tables using temporal-join-correct rolling windows. Applies point-in-time split described in Section 15.2.

17.4 Pipeline D: Baseline Build

Computes player historical baselines: serve win rate, return performance, pressure-point conversion, rally length distribution, and surface-specific tendencies. Outputs stored as versioned Parquet in `baselines/`.

17.5 Pipeline E: Model Training

Trains odds and risk models using feature tables and baseline profiles. Evaluates metrics against thresholds. Calls `publish_model.py` for two-phase validated publish.

17.6 Pipeline F: Replay Preparation

Converts historical point sequences into replay-ready event files. Assigns synthetic match IDs, replay timestamps, and partition keys.

17.7 Pipeline G: Live Replay and Streaming

Starts the replay producer, writes events to Kafka, runs Spark Structured Streaming, scores events, and writes live outputs to PostgreSQL and optional Redis.

17.8 Pipeline H: Serving API

Always-running FastAPI service. Queries PostgreSQL and Redis, formats responses, returns dashboard-ready JSON.

17.9 Pipeline I: Reporting and Evidence Export

Exports benchmark results, example match timelines, model evaluation tables, and final report figures into S3 and the repository's `reports/` folder.

17.10 Airflow DAG — Explicit Dependency Graph

CRITICAL GAP — Gap #06 addressed: DAG dependency prevents replay before model is ready

The single MVP Airflow DAG must encode the following task dependency chain:

```
ingest_task \
  >> curated_build_task \
  >> feature_build_task \
  >> baseline_build_task \
  >> train_models_task \
  >> publish_model_task \    # validates metrics, writes
     latest.json
  >> replay_prep_task \      # CANNOT start before model is
     published
  >> notify_streaming_ready_task
```

Listing 6: nightly_batch_pipeline.py — task ordering

`publish_model_task` writes `latest.json` only if validation passes (Section 14.4). The streaming startup script checks for the existence of `latest.json` before subscribing to Kafka:

```
import boto3, json, sys

s3 = boto3.client('s3')
try:
    obj = s3.get_object(Bucket='tennis-big-data',
                        Key='models/odds/latest.json')
    model_meta = json.loads(obj['Body'].read())
    MODEL_PATH = model_meta['artifact_path']
except s3.exceptions.NoSuchKey:
    print("ERROR: models/odds/latest.json not found. "
          "Run Pipeline E before starting streaming.")
    sys.exit(1)
```

Listing 7: streaming_processor.py — model readiness check

18 Streaming Correctness and Reliability Semantics

18.1 Core Streaming Assumptions

The stream is generated from historical replay so event order is controlled. Kafka partitions use `match_id` as the key so point events for the same match preserve order within a partition.

18.2 Kafka Topic Configuration

CRITICAL GAP — Gap #01 addressed: explicit Kafka topic specification

Before running `kafka-topics --create`, the following parameters must be decided and committed to `infra/ec2/kafka_setup.sh`:

```
KAFKA_BIN=/opt/kafka/bin
BOOTSTRAP=localhost:9092
TOPIC=tennis-point-events

$KAFKA_BIN/kafka-topics.sh --create \
  --bootstrap-server $BOOTSTRAP \
  --topic $TOPIC \
  --partitions 20 \           # supports up to 20 concurrent
     matches
  --replication-factor 1 \    # single-broker EC2 demo
  --config retention.ms=86400000 \   # 24-hour retention
  --config cleanup.policy=delete \
  --config max.message.bytes=1048576

# Consumer group used by Spark streaming
# (set in streaming_processor.py via
    .option("kafka.group.id","spark-tennis"))
echo "Topic $TOPIC created. Consumer group: spark-tennis"
```

Listing 8: kafka_setup.sh

Partition count rationale: 20 partitions supports up to 20 concurrent replay matches with strict ordering per match (one partition per match_id). Kafka does not allow safe partition-count changes on live topics, so this must be set correctly before the first producer run.

18.3 Spark Micro-Batch Tuning

CRITICAL GAP — Gap #02 addressed: throughput tuning parameters

The following parameters must be set before benchmarking. Expose all three as environment variables so they can be tuned without redeployment:

```
spark = SparkSession.builder \
    .appName("TennisStreamingProcessor") \
    .config("spark.sql.shuffle.partitions", "4")  \  # = vCPU
       count on m5.xlarge
    .getOrCreate()

query = (scored_stream
    .writeStream
    .trigger(processingTime="2 seconds")   # micro-batch interval
    .option("maxOffsetsPerTrigger", os.getenv("MAX_OFFSETS",
       "5000"))
    .option("checkpointLocation",
       "/mnt/checkpoints/streaming_processor")
    .option("kafka.group.id", "spark-tennis")
    .foreachBatch(write_batch)
    .start())
```

Listing 9: streaming_processor.py — tuning config

Validation: the throughput benchmark (Section 24) must demonstrate ≥ 1,000 events/second by tuning `MAX_OFFSETS` and `processingTime` together. Start with the values above and increase `MAX_OFFSETS` if throughput is below target.

18.4 Watermarking and Late Data

IMPORTANT GAP — Gap #11 addressed: watermark interval quantified

Because replay events are generated by the project, late data is rare. The watermark is set to 30 seconds. This bounds state growth for all stateful aggregations:

```
windowed = (events_df
    .withWatermark("event_ts", "30 seconds")
    .groupBy(
        F.window("event_ts", "5 minutes", "30 seconds"),
        "match_id", "player_a", "player_b"
    )
    .agg(...feature_aggregations...))
```

State TTL is bounded by the watermark plus window duration. For a 5-minute window with a 30-second watermark, state is held for at most 5.5 minutes per match. After a match ends, state is evicted automatically. Set `spark.sql.streaming.stateStore.minDeltasForSnapshot=10` to control state snapshot frequency.

18.5 Checkpoint Storage

IMPORTANT GAP — Gap #12 addressed: checkpoint location decision

Use **local EBS** for checkpointing during the demo: `checkpointLocation=/mnt/checkpoints/streaming_processor`.
Mount a dedicated EBS volume at `/mnt` so checkpoints survive instance reboots but are not lost on instance termination. **Important:** when the Spark streaming schema changes (e.g. new feature column added), the existing checkpoint must be deleted before restart. Document this in `README.md`:

```
# Run this ONLY when streaming schema changes
sudo rm -rf /mnt/checkpoints/streaming_processor
echo "Checkpoint cleared. Restart streaming_processor.py"
```

18.6 Dead-Letter Queue for Malformed Messages

CRITICAL GAP — Gap #04 addressed: streaming dead-letter handling

`dead_letter.py` wraps Spark's `from_json` so a single malformed Kafka message cannot stall the entire streaming job:

```
from pyspark.sql import functions as F
from pyspark.sql.types import StructType
```

```
def parse_with_dead_letter(raw_df, schema: StructType,
                           s3_quarantine: str) -> tuple:
    """
    Returns (good_df, bad_df).
    good_df: rows where JSON parsed successfully.
    bad_df:  rows where parse failed (written to S3 quarantine).
    """
    parsed = raw_df.select(
        F.col("offset"),
        F.col("timestamp").alias("kafka_ts"),
        F.from_json(F.col("value").cast("string"),
                    schema,
                    {"mode": "PERMISSIVE"}).alias("data"),
        F.col("value").cast("string").alias("raw_value")
    )

    good_df = parsed.filter(F.col("data.event_id").isNotNull()) \
                    .select("data.*", "kafka_ts")

    bad_df  = parsed.filter(F.col("data.event_id").isNull()) \
                    .select("raw_value", "kafka_ts",
                            F.current_timestamp().alias("quarantine_ts"))

    bad_df.write \
          .mode("append") \
          .json(s3_quarantine)   #
             s3://tennis-big-data/raw/quarantine/streaming/

    return good_df, bad_df
```

Listing 10: dead_letter.py

Call `parse_with_dead_letter` at the start of every micro-batch. Log the count of quarantined records per batch to the observability metrics (Section 22).

18.7 Idempotency and Deduplication

Every event includes a unique `event_id`. The serving writer uses PostgreSQL upserts for `current_match_state` and unique constraints on `risk_events.risk_event_id`.

18.8 State TTL and Cleanup

Streaming state evicted after watermark window expires. Redis keys carry explicit TTLs (Section 12.2). Old demo data archived to S3 by Pipeline I.

18.9 Failure Handling

- Producer retry logic on Kafka send failure;
- Kafka topic durability with 24-hour retention;
- Spark checkpoint recovery on EBS;
- database upserts for idempotent writes;
- Airflow task retries for batch jobs;
- dead-letter queue for malformed streaming records;
- logs for all failed records and failed tasks.

19 Backend Service Plan

19.1 Replay Producer

Reads prepared point-event files and publishes to Kafka. Controls replay speed, number of concurrent matches, and timestamp generation. Uses `match_id` as the Kafka message key to enforce partition routing.

19.2 Streaming Processor

Consumes Kafka events, calls `parse_with_dead_letter`, computes rolling features, applies the validated model, and writes to PostgreSQL and optional Redis.

19.3 API Service

FastAPI service exposing dashboard endpoints. No heavy analytics performed inline.

IMPORTANT GAP — Gap #14 addressed: API bearer token authentication

All API endpoints require a static bearer token validated in `api/app/middleware/auth.py`:

```
import os
from fastapi import Request, HTTPException
from starlette.middleware.base import BaseHTTPMiddleware

class BearerTokenMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        if request.url.path == "/health":
            return await call_next(request)   # health check is
                public
        token = request.headers.get("X-API-Key", "")
        if token != os.environ["API_KEY"]:
            raise HTTPException(status_code=401,
                                detail="Invalid or missing API
                                    key")
        return await call_next(request)
```

Listing 11: auth.py

The `API_KEY` value is stored in AWS SSM Parameter Store and injected at startup. The React dashboard sends the key in every request header. Document the demo key in `.env.example` as `API_KEY=${SSM:/tennis/api_key}`.

20 Frontend Build Plan

20.1 Pages

React application pages: Overview Dashboard, Live Odds Board, Match Drill-Down, Risk Explorer, System Benchmark View.

20.2 Overview Dashboard Components

Number of live matches, events processed per second, recent risk alerts, latest model version, system health cards.

20.3 Live Odds Board Components

Active match table, player win probabilities, current score, momentum indicator, last update timestamp.

20.4 Match Drill-Down Components

Odds history line chart, recent point timeline, rolling serve and return statistics, risk event markers, model explanation summary.

20.5 Risk Explorer Components

Risk event table, filters by player/match/risk level/time, risk score distribution, feature deviation details, explanation panel.

20.6 System Benchmark View

Throughput chart, latency summary, batch runtime, processed event count, resource utilization.

20.7 Frontend Refresh Policy

Live endpoints polled every 2–5 seconds. Historical pages refresh on user action only.

MINOR GAP — Gap #22 addressed: Cache-Control headers

Adding `Cache-Control` headers to FastAPI responses reduces redundant PostgreSQL reads when multiple browser tabs or demo viewers are connected simultaneously:

```
from fastapi import Response

@router.get("/api/matches/live")
async def get_live_matches(response: Response):
    response.headers["Cache-Control"] = "max-age=2,
       must-revalidate"
    return await db.fetch_live_matches()

@router.get("/api/matches/{match_id}/odds-history")
async def get_odds_history(match_id: str, response: Response):
    response.headers["Cache-Control"] = "max-age=30"
    return await db.fetch_odds_history(match_id)
```

Listing 12: routes/live.py — cache headers

21 API Contracts

21.1 Required Endpoints

```
GET  /health                                  (public, no auth)
GET  /api/matches/live                        Cache-Control: max-age=2
GET  /api/matches/{match_id}                  Cache-Control: max-age=5
GET  /api/matches/{match_id}/odds-history     Cache-Control:
   max-age=30
GET  /api/matches/{match_id}/points           Cache-Control: max-age=5
GET  /api/risk-events                         Cache-Control: max-age=5
```

```
GET  /api/risk-events/{risk_event_id}          Cache-Control:
   max-age=30
GET  /api/benchmarks/latest                    Cache-Control:
   max-age=60
GET  /api/models/current                       Cache-Control:
   max-age=60
```

21.2 API Correctness Rules

- API responses must include timestamps.
- Empty states return valid JSON, not errors.
- Database errors are handled gracefully with 500 status and logged.
- Dashboard endpoints optimized for repeated reads (indexed queries only).
- No endpoint triggers heavy Spark processing directly.
- All endpoints except `/health` require `X-API-Key` header.

22 Observability Plan

22.1 Metrics to Capture

- Kafka messages produced per second;
- Kafka messages consumed per second;
- Spark micro-batch duration;
- dead-letter message count per micro-batch;
- end-to-end event latency;
- number of active matches;
- number of risk alerts generated;
- API request latency;
- PostgreSQL row counts per table;
- batch job runtime per pipeline.

22.2 Logs to Capture

Producer logs, Kafka container logs, Spark driver and executor logs, Airflow task logs, FastAPI request/error logs, frontend deployment logs, database migration logs, dead-letter quarantine counts.

22.3 Operational Dashboards

The frontend Benchmark View serves as the main operational dashboard. Container-level metrics may be captured via `docker stats` or CloudWatch if time allows.

23 Testing Strategy

23.1 Unit Tests

- Data contract validation (each schema field);
- feature calculations (rolling window correctness, null handling);
- odds scoring functions;
- risk scoring functions (z-score thresholds);
- `parse_with_dead_letter` with intentionally malformed JSON;

- two-phase model publish — mock S3 put, assert latest.json updated only on pass;
- API response formatting;
- bearer token middleware (valid, missing, wrong token);
- frontend utility functions.

23.2 Integration Tests

- Producer to Kafka;
- Spark reading from Kafka;
- Spark writing to PostgreSQL (including JDBC batch size);
- API reading from PostgreSQL;
- frontend calling API endpoints;
- dead-letter route: malformed message reaches S3 quarantine.

23.3 End-to-End Tests

Run a small replay from S3 sample data through Kafka, Spark, PostgreSQL, API, and dashboard. Verify that a live match appears in the frontend and odds values update. Verify that a deliberately malformed event does not crash the streaming job.

23.4 Performance Tests

Vary replay speed, number of concurrent matches, and event volume. Outputs include throughput, latency, and resource utilization. **Discard the first 60 seconds of metrics** as the Spark JIT, Kafka consumer rebalance, and JVM GC warm-up window (see Section 24).

24 Benchmarking Plan

24.1 Benchmark Scenarios

Scenario	Description
Baseline replay	Small number of matches, low replay rate
Moderate load	Hundreds of concurrent simulated matches
High load	Maximum sustainable replay rate for selected EC2 instance
Failure recovery	Restart streaming job, verify checkpoint recovery
Serving stress	Repeated dashboard/API reads during replay

Table 6: Benchmark scenarios

24.2 Metrics Reported

Events produced/second, events processed/second, average and p95 streaming latency, API response latency, batch job runtime, model metrics, alerts generated.

24.3 Benchmark Success Criteria

- System processes a live replay stream end-to-end.
- Dashboard updates within 5 seconds of event production.
- Risk events stored and displayed correctly.
- Benchmark evidence exported to `results/benchmarks/`.

IMPORTANT GAP — Gap #21 addressed: warm-up period

All benchmark measurement windows must **exclude the first 60 seconds** after streaming job startup. This accounts for Spark JIT compilation, Kafka consumer group rebalancing, and JVM GC ramp-up. Document the warm-up window in `results/benchmarks/README.md` so graders understand the methodology. The benchmark script must record:

- `warmup_seconds`: 60
- `measurement_start`: timestamp after warm-up
- `measurement_duration`: configurable (recommend 300 seconds)

25 Analytical Output Plan

25.1 Required Analytical Questions

- How do win probabilities change during a simulated match?
- Which features contribute most to odds movement?
- Which players or matches generate high risk scores?
- How does system latency change as event throughput increases?
- How do batch and streaming layers interact?

25.2 Required Charts and Tables

Live odds over time, momentum timeline, risk score distribution, feature deviation table, throughput and latency chart, model evaluation table, architecture diagram.

25.3 Business Interpretation Requirement

Every analytical output must include a plain-English interpretation. A high risk score must be explained as a statistical anomaly relative to baseline, not as proof of wrongdoing.

26 Report, Presentation, and Deliverables Plan

26.1 Required Final Deliverables

GitHub repository with code and documentation; Docker Compose files for both EC2 instances and local dev; S3 data lake structure with zone manifests; working Kafka replay with explicit topic config; Spark batch and streaming jobs with dead-letter handling; trained model artifacts with two-phase publish; PostgreSQL serving schema with indexes and archival; FastAPI backend with auth; React frontend; benchmark results with warm-up methodology documented; final report; final presentation.

26.2 Project Report Structure

Problem statement; architecture; data engineering design; Kafka and streaming configuration; modeling methodology (unsupervised anomaly approach); implementation details; benchmark results; dashboard screenshots; limitations and future work.

26.3 Presentation Slide Structure

1. Title and motivation

2. Problem statement
3. Architecture overview
4. Data lake and pipelines
5. Streaming demo flow
6. Model design
7. Dashboard screenshots
8. Benchmark results
9. Limitations
10. Conclusion

26.4 Executive Audience Standard

The final presentation explains the system clearly to both technical and business audiences. It emphasizes: what was built, why the architecture is appropriate, what tradeoffs were made, and what evidence proves the system works.

27 Milestones

27.1 Milestone 1: Data Foundation

Upload raw datasets to S3, define data contracts with JSON schemas, implement cleaning scripts with schema validation, write zone manifests.

27.2 Milestone 2: Offline Modeling

Build features with temporal correctness, compute baselines, train initial odds and risk models, validate against thresholds, publish via two-phase publish.

27.3 Milestone 3: Live Replay

Create Kafka topic with explicit config, implement producer, verify point events published and consumed, verify dead-letter handling.

27.4 Milestone 4: Serving Layer

Create PostgreSQL schema with indexes and archival, implement writers with JDBC batch size, validate current match state updates.

27.5 Milestone 5: Frontend MVP

Build live match overview and odds board with cache headers.

27.6 Milestone 6: Risk Explorer

Add risk-event API and frontend risk explorer page.

27.7 Milestone 7: Benchmark and Report Assets

Run performance tests with warm-up excluded, export charts and tables.

27.8 Milestone 8: Final Demo Readiness

Run full end-to-end demo, clean repository, finalize report and presentation.

28 MVP vs. Stretch Goals

28.1 MVP

S3 data lake with Iceberg on curated/features zones (Hadoop catalog); one Kafka topic with explicit partition config; one Spark streaming job with dead-letter handling; basic odds model with validation gate; basic risk score (z-score baseline); PostgreSQL serving tables with indexes and archival; FastAPI endpoints with auth; React dashboard with cache headers; benchmark evidence with warm-up documented.

28.2 Stretch Goals

Redis caching; Airflow multi-DAG orchestration; advanced model comparison; model version dashboard; larger synthetic load tests; richer match drill-down visualizations; automatic report exports; CloudWatch metrics integration.

29 Explicit Coding Order

1. Define data contracts and write JSON schema files
2. Create S3 folder layout with `create_layout.sh` and zone manifests
3. Write IAM role policies for EC2-1 and EC2-2
4. Configure SSM Parameter Store with all secrets
5. Build local sample data loader with schema validation
6. Build cleaning pipeline with Iceberg Hadoop catalog config
7. Build feature pipeline with temporal correctness
8. Create PostgreSQL schema with indexes and archival job
9. Configure Kafka topic with `kafka_setup.sh`
10. Build replay producer
11. Build Spark streaming read from Kafka with dead-letter handling
12. Add rolling feature computation with watermark
13. Add odds and risk scoring with model readiness check
14. Add two-phase model publish with validation gate
15. Encode Airflow DAG dependencies (E $\rightarrow$ F)
16. Write outputs to PostgreSQL with JDBC batch size
17. Build FastAPI endpoints with auth middleware and cache headers
18. Build React dashboard
19. Run benchmarks with warm-up exclusion
20. Finalize report and demo

30 Code Quality, Reproducibility, and CI

30.1 Quality Controls

- Consistent Python formatting (ruff or black);
- SSM-backed environment variable management;
- schema validation on all raw ingestion;
- unit tests for all core logic including dead-letter and auth;
- clear README with Quick Start, schema change instructions, checkpoint clear procedure;
- reproducible Docker Compose commands.

30.2 Reproducibility Rules

- Raw data must not be manually edited;
- every pipeline output reproducible from source inputs;
- model artifacts include version, training timestamp, and metrics;
- benchmark runs record configuration values including warm-up window.

30.3 CI Pipeline

IMPORTANT GAP — Gap #17 addressed: concrete CI toolchain

GitHub Actions (`.github/workflows/ci.yml`) with three jobs:

```
name: CI

on: [push, pull_request]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.11"}
      - run: pip install ruff
      - run: ruff check spark/ api/ producer/

  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.11"}
      - run: pip install -r requirements-test.txt
      - run: pytest tests/unit/ --tb=short -q

  compose-validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker compose -f docker-compose.processing.yml
          config
      - run: docker compose -f docker-compose.serving.yml config
      - run: docker compose -f docker-compose.dev.yml config
```

Listing 13: ci.yml

This runs in under 3 minutes and catches formatting failures, broken unit tests, and invalid Docker Compose configs before they reach the EC2 instances.

31 Environment Variables

All secrets stored in AWS SSM Parameter Store. Non-secret configuration stored in `.env` files on each EC2 instance, bootstrapped from `.env.example`.

```
# AWS
```

```
AWS_REGION=us-east-1
S3_BUCKET=tennis-big-data

# Kafka (EC2-1 internal)
KAFKA_BOOTSTRAP_SERVERS=localhost:9092
KAFKA_TOPIC=tennis-point-events
KAFKA_PARTITIONS=20
KAFKA_CONSUMER_GROUP=spark-tennis

# Spark tuning
SPARK_MAX_OFFSETS=5000
SPARK_TRIGGER_INTERVAL=2 seconds
SPARK_SHUFFLE_PARTITIONS=4

# PostgreSQL (read from SSM on EC2-2, referenced by private IP on
   EC2-1)
POSTGRES_HOST=${SSM:/tennis/postgres_host}
POSTGRES_PORT=5432
POSTGRES_DB=tennis_serving
POSTGRES_USER=tennis_user
POSTGRES_PASSWORD=${SSM:/tennis/postgres_password}

# Redis
REDIS_HOST=${SSM:/tennis/redis_host}
REDIS_PORT=6379

# Models
MODEL_ODDS_PATH=s3://tennis-big-data/models/odds/latest.json
MODEL_RISK_PATH=s3://tennis-big-data/models/risk/latest.json

# API
API_BASE_URL=http://ec2-serving-elastic-ip:8000
API_KEY=${SSM:/tennis/api_key}

# Benchmark
BENCHMARK_WARMUP_SECONDS=60
BENCHMARK_MEASUREMENT_SECONDS=300
```

Listing 14: .env.example

32 Milestone Gates

32.1 Gate 1

Data can be read from S3, schema-validated, and cleaned into Iceberg-managed curated Parquet.

32.2 Gate 2

Offline features (with temporal correctness) and model artifacts (passing validation thresholds) are generated and published via two-phase publish.

32.3 Gate 3

Kafka topic created with explicit config; replay and Spark streaming produce scored outputs; dead-letter handling verified with malformed test message.

32.4 Gate 4

Dashboard displays live odds and risk events from serving tables; archival job confirmed running; API auth verified.

33 Course Presentation Strategy

The presentation focuses on architecture first, then demonstrates evidence. The strongest story is that one event stream powers two analytical products: live odds and integrity monitoring. The system is explained as a practical Lambda-style architecture where S3 stores durable data, EC2-1 performs batch and streaming computation, and EC2-2 serves low-latency application views.

Emphasize the gap-resolution decisions as evidence of engineering rigor: explicit Kafka topic sizing, validated model publish, dead-letter handling, IAM separation, and temporal feature correctness.

34 Limitations

- The system uses simulated live streams, not actual live tennis feeds.
- Risk scores are statistical indicators, not evidence of misconduct.
- The two-EC2 deployment is not production scale.
- Model accuracy depends on available historical data quality.
- Synthetic amplification is useful for systems testing but does not create new real-world evidence.
- The Hadoop Iceberg catalog is not suitable for concurrent multi-writer production workloads; for production, AWS Glue catalog would replace it.
- The static bearer token API auth is sufficient for demo purposes only; production would use OAuth 2.0 or API gateway-level auth.

Appendix A: AI Coding Agent Execution Notes

When using an AI coding assistant, request small, testable components rather than the full system at once. Each generated module must be validated with unit tests or sample input/output checks. The AI assistant must not change architecture decisions unless the team explicitly approves.

Recommended sequence: data contracts and schemas; cleaning functions; producer logic; Spark transformations (batch then streaming with dead-letter); API endpoints with auth; frontend components; deployment scripts.

Appendix B: Architecture and Flow Diagrams

B.1 Corrected High-Level System Architecture

Amazon S3 Data Lake
raw / cleaned (Iceberg) / features (Iceberg) / baselines / models / stream_archive / results

↕ features, models, outputs ↕ scored results

EC2-1 Processing Node	⟶	**EC2-2 Serving/App Node**
Kafka + Spark + Airflow + ML	scored outputs	PostgreSQL + Redis + API + React

↑ Replay Producer ↓ Dashboard users (Live Odds + Risk Explorer)

B.2 Corrected Data Lifecycle and Storage Flow

1. Raw data uploaded to S3 raw zone (schema-validated).
2. Cleaning jobs produce curated Iceberg tables.
3. Feature jobs produce feature Iceberg tables (temporally correct).
4. Baseline jobs produce player profiles.
5. Model training jobs publish artifacts via two-phase validated publish.
6. Replay jobs create live-like events.
7. Streaming outputs archived back to S3 stream_archive zone.

B.3 Offline Training and Model Publication Flow

Cleaned Data (S3 Iceberg) → Feature Build (Spark Batch) → Baseline Build → Model Training → Validation Gate (AUC ≥ 0.55) → Publish to S3 + update `latest.json` → Airflow notifies replay prep.

B.4 Real-Time Streaming and Scoring Flow

Producer → Kafka (`tennis-point-events`, 20 partitions) → Spark Structured Streaming (dead-letter split) → rolling features (watermark 30s) → model load from `latest.json` → PostgreSQL upserts (JDBC batch 500) → optional Redis TTL cache.

B.5 MVP vs. Stretch Architecture Scope

Area	**MVP**	**Stretch**
Data Lake	S3 zones + Iceberg (Hadoop catalog) + zone manifests	Glue catalog, richer partitioning
Streaming	One Kafka topic, 20 partitions, dead-letter	Multiple topics, retry queues
Processing	One Spark streaming job	Separate odds and risk streams
Models	Baseline odds/risk with validation gate	Advanced model comparison
Serving	PostgreSQL + indexes + archival	PostgreSQL + optimized Redis cache
Frontend	Dashboard MVP + cache headers	Rich drill-down and explainability
Orchestration	Single Airflow DAG with explicit dependencies	Multi-DAG pipeline suite
Auth	Static bearer token	OAuth 2.0
Secrets	AWS SSM Parameter Store	HashiCorp Vault

Table 7: MVP vs. stretch architecture comparison

B.6 Gap Resolution Summary

Gap	Title	Severity	Section Addressed
#01	Kafka topic config	Critical	Section 18.2
#02	Spark micro-batch tuning	Critical	Section 18.2
#03	PostgreSQL write bottleneck	Critical	Section 12.1
#04	Dead-letter queue	Critical	Section 18.5
#05	IAM role separation	Critical	Section 8.3
#06	Airflow DAG dependency	Critical	Section 17.10
#07	Point-in-time feature correctness	Critical	Section 15.2
#08	Ingestion trigger	Important	Section 17.1
#09	Iceberg catalog backend	Important	Section 7.2
#10	EC2-1 OOM risk	Important	Section 8.2
#11	Watermark interval	Important	Section 18.3
#12	Checkpoint location	Important	Section 18.4
#13	Two-phase model publish	Important	Section 14.4
#14	FastAPI auth	Important	Section 19.3
#15	odds_history size	Important	Section 12.3
#16	Anomaly training strategy	Important	Section 13.2
#17	CI toolchain	Important	Section 30.3
#18	Shared secrets management	Important	Section 8.4
#19	S3 zone manifests	Minor	Section 10.2
#20	Model evaluation thresholds	Minor	Section 14.4
#21	Benchmark warm-up period	Minor	Section 24.3
#22	API cache headers	Minor	Section 20.7
#23	Docker Compose network	Minor	Section 8.1
#24	Parquet schema evolution	Minor	Section 9.4

Table 8: Complete gap resolution index